

Online Library Python Program – Full Line-by-Line Explanation

This PDF explains the complete Online Library Python project line-by-line. It includes:

- Functions
- Variables
- File handling
- Login system
- Book logging system
- Menu system
- Error handling

1. Importing OS Module

```
import os
```

Imports Python's built-in os module for handling files, folders, and paths.

2. Credentials File

```
CRED_FILE = r"c:\Users\Akshaj Agarwal\pyprog002\credentials.txt"
```

Stores the location of the credentials file containing usernames and passwords.

3. Books File

```
BOOKS_FILE = r"c:\Users\Akshaj Agarwal\pyprog002\books\available_books.txt"
```

Stores the location of the available books list.

4. Folder Variables

```
LOG_FOLDER, TRIES_FOLDER, MAX_TRIES = "book_logs", "user_tries", 5
```

Creates variables for book logs folder, login tries folder, and maximum login attempts.

5. Create Folders

```
os.makedirs(LOG_FOLDER, exist_ok=True)  
os.makedirs(TRIES_FOLDER, exist_ok=True)
```

Creates required folders automatically if they do not already exist.

6. get_books() Function

```
def get_books():
    try:
        with open(BOOKS_FILE) as f:
            return [l.strip() for l in f if l.strip() and l[0].isdigit()]
    except:
        return []
```

Reads the available books file and returns a clean list of books.

7. get_tries() Function

```
def get_tries(name):
    try:
        with open(os.path.join(TRIES_FOLDER, f"{name}_tries.txt")) as f:
            return int(f.read())
    except:
        return 0
```

Reads how many wrong login attempts a user has made.

8. save_tries() Function

```
def save_tries(name, n):
    try:
        with open(os.path.join(TRIES_FOLDER, f"{name}_tries.txt"), "w") as f:
            f.write(str(n))
    except:
        pass
```

Saves the number of wrong login attempts.

9. check_pwd() Function

```
def check_pwd(name, pwd):
    try:
        with open(CRED_FILE) as f:
            lines = f.readlines()
            for i, line in enumerate(lines):
                if f"USERNAME: {name}" in line and i+1 < len(lines) and f"PASSWORD: {pwd}" in lines[i+1]:
                    return True
    except:
        pass
    return False
```

Checks whether the username and password entered by the user are correct.

10. make_account() Function

```
def make_account(name, pwd):
    try:
        with open(CRED_FILE) as f:
            if f"USERNAME: {name}" in f.read():
                print("Already exists!")
                return
    except:
        pass
```

Creates a new user account if the username does not already exist.

11. log_book() Function

```
def log_book(name, book, fully_read):
    try:
        with open(os.path.join(LOG_FOLDER, f"{name}_books.txt"), "a") as f:
            f.write(f"{book} - {'Fully' if fully_read else 'Partially'} Read\n")
        print("Book saved!")
    except:
        print("Could not save.")
```

Saves information about books read by the user.

12. show_menu() Function

```
def show_menu(name, books):
    while True:
        print("1. See All Books")
        print("2. Log a Book")
        print("3. See My Books")
        print("4. Exit")
```

Displays the main menu after login.

13. main() Function

```
def main():
    print("ONLINE LIBRARY")
    pick = input("(1) Login or (2) Create Account? ")
```

Controls the full flow of the program.

14. Program Starter

```
if __name__ == "__main__":
    main()
```

Starts the program when the Python file is run directly.

Program Flow Summary

```
Start
↓
Create folders
↓
Create account or login
↓
Check password
↓
Load books
↓
Show menu
↓
Save books or view books
↓
Exit
```

This project demonstrates important Python concepts:

- Functions

- File handling
- Loops
- Conditions
- Error handling using try/except
- User authentication
- Data storage